Retrieval-Pretrained Transformer: Long-range Language Modeling with Self-retrieval

Ohad Rubin **Jonathan Berant**
The Blavatnik School of Computer Science, Tel Aviv University
{ohad.rubin,joberant}@cs.tau.ac.il

arXiv:2306.13421v2 [cs.CL] 21 Jul 2024

Abstract

Retrieval-augmented language models (LMs) have received much attention recently. However, typically the retriever is not trained jointly as a native component of the LM, but added post-hoc to an already-pretrained LM, which limits the ability of the LM and the retriever to adapt to one another. In this work, we propose the *Retrieval-Pretrained Transformer* (RPT), an architecture and training procedure for jointly training a retrieval-augmented LM from scratch and apply it to the task of modeling long texts. Given a recently generated text chunk in a long document, the LM computes query representations, which are then used to retrieve earlier chunks in the document, located potentially tens of thousands of tokens before. Information from retrieved chunks is fused into the LM representations to predict the next target chunk. We train the retriever component with a semantic objective, where the goal is to retrieve chunks that increase the probability of the next chunk, according to a reference LM. We evaluate RPT on four long-range language modeling tasks, spanning books, code, and mathematical writing, and demonstrate that RPT improves retrieval quality and subsequently perplexity across the board compared to strong baselines.

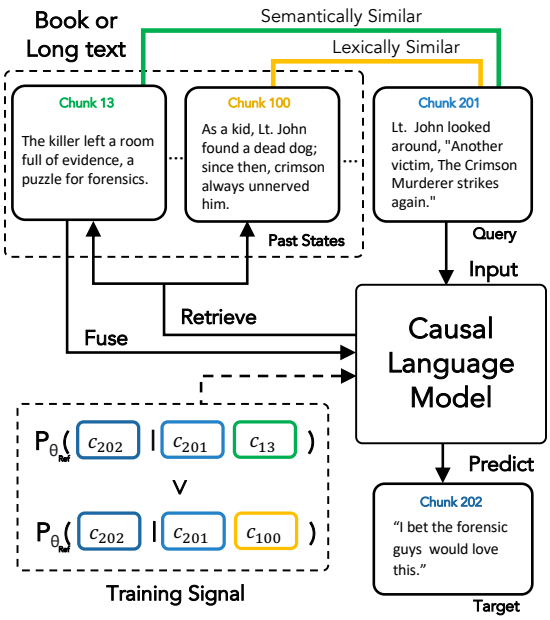

Figure 1: Retrieval-Pretrained Transformer (RPT) is a language trained from scratch with a native retrieval ability that can be applied to long texts (e.g., books). RPT takes a chunk of text as input, retrieves semantically-relevant chunks from the past to better predict the next chunk, and fuses these retrieved chunks into its representations. On top of a standard LM loss, the retriever is trained to retrieve chunks that increase the probability of the next chunk according to a *reference LM*.

1 Introduction

Large language models (LMs) have had immense success recently (Brown et al., 2020; Chowdhery et al., 2022; Zhang et al., 2022; Touvron et al., 2023), becoming a useful tool across disciplines. However, their success comes at a computational cost, due to increasing parameter counts for storing world knowledge (Fedus et al., 2022) and growing context lengths that enable access to distant information, but incur a quadratic complexity penalty. Retrieval-augmented language modeling (RALM) alleviates this cost (Khandelwal et al., 2020; Yogatama et al., 2021; Borgeaud et al., 2022; Ram et al., 2023), as precise retrieval of relevant information can reduce memory and computation requirements. Moreover, RALM is beneficial for factuality, freshness and generalization without necessitating retraining, simply by swapping the retrieval index (Guu et al., 2020; Lewis et al., 2020; Huang et al., 2023).

However, past work on RALM has by and large *not* trained the retriever as a first-class component of the LM. In some cases (Khandelwal et al., 2020;

Yogatama et al., 2021; Borgeaud et al., 2022), the retriever was used only at test time, or remained fixed throughout training, preventing it from adapting to the LM generator. In other cases, the retriever component was jointly trained but only after a separate pretraining phase for both the retriever and LM (Sachan et al., 2021; Izacard et al., 2022b; Jiang et al., 2022; Bertsch et al., 2023). Thus, the retriever was not pre-trained from scratch with the LM, and only a fraction of the training budget was allocated for joint training.

Recently, Zhong et al. (2022) presented a retrieval-augmented LM that trains a retriever from scratch jointly with the LM, but (a) the retriever was trained to exploit *lexical* information only, and (b) the retrieved information was not fused at the *representation level* back into the LM.

In this work, we present the *Retrieval-Pretrained Transformer (RPT)*, a retrieval-augmented LM, where the retriever is a first-class component, trained jointly from scratch with the LM. RPT relies on two technical contributions. First, on the architecture side (see Fig. 1), input representations for the retriever are computed from the LM representations themselves (a concept we dub *self-retrieval*), and retrieved representations are fused back into the LM decoder for making next word predictions. Second, we train the retriever with an *auxiliary loss function* that encourages retrieving text fragments that increase the probability of generating the subsequent text. Specifically, given a recently-generated chunk c_t, the retriever is trained to retrieve chunks c_i that increase the probability of $p_{\text{scoring}}(c_{t+1} \mid c_i, c_t)$ according to a reference *scoring LM*. Fig. 1 provides an illustrative example for a case where a crime scene is described, and a scoring LM shows the benefit of retrieving a chunk thousands of tokens away (chunk 13) compared to lexical retrieval, which leads to a chunk that is only superficially related (chunk 100). Unlike existing retrieval-augmented models that use an auxiliary encoder for retrieval (Izacard and Grave, 2021a; Izacard et al., 2022b; Sachan et al., 2021), RPT is able to leverage its internal hidden states for retrieval after a *single* pre-training stage, greatly simplifying joint training.

We apply RPT to the problem of modeling long documents, such as books, articles and code, as those are naturally occurring examples of long-form content, where the entire index can be held within memory in a forward-pass.

We evaluate RPT on four language modeling tasks and find that it improves perplexity across all tasks, outperforming prior work (Hutchins et al., 2022; Wu et al., 2022) as well as strong baselines (Borgeaud et al., 2022; Zhong et al., 2022). Moreover, we show that RPT retrieves high-quality chunks compared to retrievers that rely on lexical information. Based on our empirical findings, we argue RPT can pave the way toward a next generation of pre-trained LMs, where large corpora are used during pre-training, resulting in a language models where retrieval is a strongly embedded component. Our code is publicly available at `https://github.com/OhadRubin/RPT`.

2 Background

To situate our contribution, we review relevant recent RALM work. We extend this to more related work in §6.

Early work on RALMs, such as kNN-LM (Khandelwal et al., 2020) used retrieval to improve language modeling by interpolating the next-word distribution produced by the LM with a distribution proposed through a *test-time-only* retrieval mechanism. Borgeaud et al. (2022) later proposed Chunked Cross-Attention (CCA), where retrieval is performed also at training time, and retrieval results are deeply fused into the representations produced by a Transformer decoder through attention. However, the retriever was trained separately and kept fixed during training, which prevented it from adapting to the LM over the course of training.

TRIME (Zhong et al., 2022), like this work, trained a retrieval-augmented LM from scratch where the retriever component and the decoder LM are trained jointly. Our work differs from TRIME in two aspects: First, TRIME, like kNN-LM, incorporates information from the retriever in a shallow manner through distribution interpolation, while we adopt CCA as a deeper fusion mechanism. Second, TRIME takes advantage of lexical clues for supervising the retriever, that is, given a query, the TRIME retriever learns to retrieve contexts that will lead to generating the same token as the query. We, on the other hand, use a scoring LM to evaluate what text chunks are relevant for increasing the probability of the chunk being generated, which leads to more semantic retrieval. This is similar to EPR (Rubin et al., 2022), which used this idea for learning to retrieve prompts for in-context learning, and perplexity distillation in Atlas (Izacard et al.,

2022b). However, Atlas does not train the retriever and LM from scratch and is an encoder-decoder model, more suitable for knowledge-intensive tasks. We, conversely, train from scratch and use a decoder model, more suitable for modeling long texts.

3 Retrieval-Pretrained Transformer

Problem Setup Like RETRO (Borgeaud et al., 2022), RPT is a chunk-wise retrieval-augmented LM that divides the input sequence into chunks for retrieval. Specifically, given a sequence of L input tokens, $(x_1, x_2, \ldots, x_L)$, we partition it into a sequence of $\ell = \frac{L}{m}$ non-overlapping chunks of length m, denoted by $\mathcal{C} = (c_1, c_2, \ldots, c_\ell)$. For every possible *query* chunk, $c^{\mathbf{q}} = c_i$, the model will retrieve a subset of at most $K \ll \ell$ chunks, $\mathcal{R}(c^{\mathbf{q}}) \subset \mathcal{C}^{<i} = (c_1, c_2, ..., c_{i-w})$, where $\mathcal{C}^{<i}$ is the set of *retrievable* chunks for c_i, which excludes the w chunks to which it already has access to through causal self-attention. The goal is to learn a model that retrieves a chunk subset, $\mathcal{R}(c^{\mathbf{q}})$, that increase the probability of autoregressive generation of the *target chunk* $c^{\mathbf{t}} = c_{i+1}$.

We present our method in two parts. First, our architecture (§3.1), which leverages CCA to fuse retrieved representations into the LM, but adds a learned retriever component. Second, we present the training method (§3.2-§3.3), where the retriever is trained to retrieve chunks useful for generating a future chunk according to a reference LM.

3.1 Model Architecture

Fig. 2 illustrates our architecture, where the input has 45 input tokens divided into 9 chunks, and causal self-attention is applied over $w = 3$ chunks (15 tokens). The left side depicts the decoder stack (*"reader"*), and the right side the retriever. The reader is split into two, where the bottom $\frac{n_{\text{layers}}}{2}$ layers (*lower decoder*) are standard Transformer decoder layers that take w chunks as input and output representations that will be used by the retriever and the top decoder layers.

The top $\frac{n_{\text{layers}}}{2}$ layers (upper decoder) use Chunked Cross-Attention (CCA) to fuse information from the top-K neighbor chunks retrieved by the retriever back into the LM. We use standard CCA layers from RETRO (Borgeaud et al., 2022), where for each one of the ℓ chunks, queries are the m token representations of that chunk output by causal attention, and the keys and values are the token representations for the top-K neighbor chunks output by the retriever.[1]

Next, we describe the retriever component, along with a neighbor gating mechanism for modulating the effect of retrieved representations.

Retriever The retriever takes as input the representations output by the lower decoder and produces a similarity score for every pair of chunks. Given a *query chunk* $c^{\mathbf{q}}$, the *query-based score* for each retrievable chunk c is $s_{\mathbf{q}}(c) = \langle W_Q \mathbf{c}^{\mathbf{q}}, W_K \mathbf{c} \rangle$, where $W_Q, W_K \in \mathbb{R}^{d \times d}$ are learned linear projections, and $\mathbf{c}^{\mathbf{q}}$ and $\mathbf{c}$ are chunk representations.

For an m-token long chunk c, we compute its representation $\mathbf{c}$ by applying bidirectional attention over the chunk tokens, followed by mean-pooling across the time dimension. This maintains causality, as these representations are only used during the prediction of the next chunk.

Once scores for all pairs of chunks are computed, the *retrieved neighbor chunks* $\mathcal{R}(c^{\mathbf{q}})$, for each query chunk, $c^{\mathbf{q}}$, consists of its top-K highest-scoring retrievable chunks. Then, for each chunk $c_j \in \mathcal{R}(c^{\mathbf{q}})$, we concatenate the representations of the succeeding chunk c_{j+1} to provide additional context, and the final representation for all neighbors of all chunks is given by a tensor $C \in \mathbb{R}^{\ell \times K \times 2m \times d}$.[2]

Overall (and unlike methods like TRIME and kNN-LM), the retriever is an integral part of the LM, where the lower decoder computes representations for the retriever (which we dub *self-retrieval*), and the upper decoder consumes representations produced by the retriever.

Neighbor gating We add a neighbor gating mechanism to softly select neighbor representations that are useful for fusing into the upper decoder. Let $C_{i,k} \in \mathbb{R}^{2m \times d}$ be the token representations for the k'th neighbor of chunk c_i. We mean-pool across the time dimension to obtain a vector $\hat{\mathbf{c}}_{i,k}$ for each neighbor chunk. Then, we enrich the neighbor representation of each chunk by applying causal attention – a neighbor chunk representations $\hat{\mathbf{c}}_{i,k}$ attends to chunks that precede it or to neighbors of the same chunk c_i that are ranked higher. Finally, for each chunk we obtain the *gated retrieved representation* by multiplying the augmented representations by a gating score:

[1] For full details of CCA, see Borgeaud et al. (2022).

[2] Similar to RETRO, token representations of retrieved chunks are also augmented through cross-attention over tokens of the query chunk, $c^{\mathbf{q}}$.

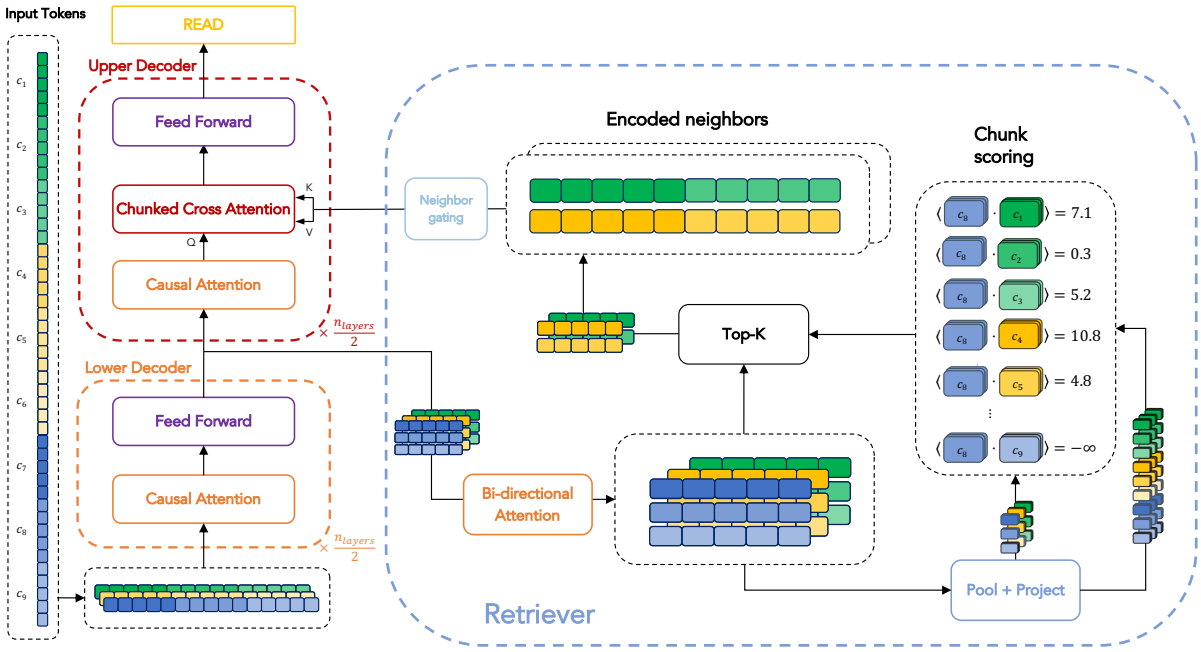

Figure 2: The architecture of the *Retrieval-Pretrained Transformer*, where an input of 45 tokens is shown, consisting of 9 chunks, and causal self-attention is applied over 15 tokens. The left side shows the decoder stack, where the bottom $\frac{n_{\text{layers}}}{2}$ are standard Transformer decoder layers, and the top $\frac{n_{\text{layers}}}{2}$ layers also include chunked cross-attention layers that fuse information from retrieved chunks. The right side shows the retriever, which takes a chunk and retrieves the highest-scoring K chunks that appeared earlier in the document.

$C^{\mathbf{g}}_{i,k} = \max\{\eta, \sigma(\frac{\mathbf{w}_{\text{ng}}\hat{\mathbf{c}}_{i,k}}{d})\} \cdot C_{i,k}$ where $\mathbf{w}_{\text{ng}}$ is a learned parameter vector, η is a small value meant to maintain gradient flow,[3] and σ is the sigmoid activation. Finally, in the upper decoder, when CCA is performed, the keys and values are $C^{\mathbf{g}}_{i,k}$.

3.2 Supervision Signal

For each query chunk $c^{\mathbf{q}} = c_i$, we want to identify neighbor chunks that will be helpful for generating $c^{\mathbf{t}} = c_{i+1}$, and use those neighbor chunks as supervision signal for the retriever. Similar to Rubin et al. (2022), we can exploit the fact that we are producing *training data* and use information from $c^{\mathbf{t}}$ itself to produce such a score. Unlike Zhong et al. (2022), who use lexical clues alone, we will use an independent *scoring LM* for this purpose.

Scoring every chunk w.r.t to all preceding chunks is quadratic in the number of chunks in a document, and thus computationally difficult. Thus, we use a simple, BM25 unsupervised retriever (Robertson and Zaragoza, 2009) that takes as input the concatenation of the chunks $(c^{\mathbf{q}}, c^{\mathbf{t}}) = (c_i, c_{i+1})$ and returns a set of candidates neighbor chunks, $\bar{\mathcal{R}} \subset \mathcal{C}(c^{\mathbf{q}})$, which have high lexical overlap with the current and subsequent chunk. This retriever has access to the tokens that need to be generated by the LM, which is allowed at training time.

Let $\hat{g}$ be an independently-trained LM, and let $\bar{c}_j$ be the concatenation (c_j, c_{j+1}). We compute a score $s_{\mathbf{t}}(\bar{c}_j)$ that reflects whether the information in $\bar{c}_j$ is more useful for decoding $c^{\mathbf{t}}$ compared to chunks that are close to $c^{\mathbf{q}}$. Specifically, the *target-based score* for a candidate chunk is

$$s_{\mathbf{t}}(\bar{c}_j) = \log \frac{\text{Prob}_{\hat{g}}(c^{\mathbf{t}} \mid c_j, c_{j+1}, c^{\mathbf{q}})}{\text{Prob}_{\hat{g}}(c^{\mathbf{t}} \mid c_{i-2}, c_{i-1}, c^{\mathbf{q}})}.$$

This score is positive when information in $\bar{c}_j$ is more useful for decoding $c^{\mathbf{t}}$ than information in the preceding two chunks (c_{i-2}, c_{i-1}).

We apply this scoring function to all chunks, and define for each query chunk $c^{\mathbf{q}}$ the set of *positive chunks* $\mathcal{R}^{\mathbf{q}}_{\text{pos}}$, which includes candidates for which $s_{\mathbf{t}}(\cdot) > 0$. This should result in helpful chunks, as each candidate chunk is at least as good as the local context. With this ordering at our disposal, we can apply standard retrieval training methods.

3.3 Training

To train the parameters of the retriever component, we adapt the widely-used LambdaRank loss (Burges et al., 2006). The loss for each query chunk

[3] We set $\eta = 0.1$ in all of our experiments.

$c^{\mathbf{q}}$ (w.r.t its retrievable chunks) is:

$$L_{\text{ret}}(c^{\mathbf{q}}) = \sum_{\{j,l:\bar{c}_l \in \mathcal{R}^{\mathbf{q}}_{\text{pos}}, s_{\mathbf{t}}(\bar{c}_l) > s_{\mathbf{t}}(\bar{c}_j)\}} \lambda_{jl} \max\left(0, \tau - (s_{\mathbf{q}}(c_l) - s_{\mathbf{q}}(c_j))\right)$$

where τ is a margin hyper-parameter, and λ_{jl} is the LambdaRank scaling that considers the relative ranking of each candidate. This loss is non-zero when for some pair of candidates, the target-based score disagrees (with margin τ) with the ranking of the query-based score for candidates in $\mathcal{R}^{\mathbf{q}}_{\text{pos}}$. Optimizing this loss function allows RPT to distinguish between relevant and irrelevant chunks. Our final loss is $L_{\text{LM}} + \alpha_{\text{ret}} L_{\text{ret}}$, where L_{LM} is the standard LM loss and α_{ret} is the retrieval loss coefficient, increased linearly in the first 100K steps. We also increase τ linearly during training.

3.4 Important Implementation Details

Scheduled sampling To reduce train-test mismatch, we apply scheduled sampling (Bengio et al., 2015) during training. Namely, after computing the top-K neighbor chunks, we use these neighbors with probability $1 - p_{\text{ss}}$, and with probability p_{ss} the top-K scoring candidates from $\mathcal{R}^{\mathbf{q}}_{\text{pos}}$ as input for CCA. We anneal p_{ss} from 1 to 0 during the first 90% of training with a cosine schedule. This allows the model to gradually learn to use its own predictions. We report the effect of this in §5.3.

Sliding window attention at training and inference time As described in §3, the decoder takes as input w chunks, each with m tokens as input, and applies causal attention over them. In practice, to give the first tokens access to past tokens, we use the sliding-window attention mechanism (Dai et al., 2019; Beltagy et al., 2020; Ivgi et al., 2023), where the number of tokens in a window is 2,048 and the stride is 1,024. Thus, the input to each window is 2,048 tokens and the output are the representations for the last 1,024 tokens, which use the keys and values of the previous 1,024 tokens for contextualization.

At inference time a similar procedure is applied. We compute and cache the key and value representations for segments of 1,024 tokens, using these as context for generating or estimating the probability of the next segment.

Retrieval at inference time During training we encode in each batch sequences of length 16K and retrieve chunks from those encoded 16k tokens.

Name	Tokens (Train/Test)	Median Length
ArXiv	12,000 / 16	16,368
CodeParrot	5,000 / 5	29,269
PG19	3,000 / 9	82,659
Books3	25,000 / 35	113,496

Table 1: Number of tokens (in millions) for each dataset and median document length.

However, at inference time the retriever provides access to *all* tokens from the start of the document, where we store the key and lower-decoder representations in a Faiss (Douze et al., 2024) index on the CPU. For each chunk, we query the index using the chunk's query representations and retrieve the top-K lower-decoder representations with the highest dot product.

Additional details At training time we use sequences of length $L = 16,384$ tokens, which are split into 4 devices, each consuming $4,096$ tokens. As mentioned, the decoder stack takes $2,048$ tokens as input (in a sliding window approach), which contains $\ell = 32$ chunks of length $m = 64$. We employ Rotary Positional embedding (Su et al., 2024), and train all models for 500K steps on a TPUv4-64, with an effective batch size of 2^{17} tokens resulting in a total training budget of 65 billion tokens.

For all models trained, we use the GPT-NeoX (Black et al., 2022) tokenizer, which was trained on the Pile (Gao et al., 2020) and covers the domains we evaluate on (see §4). As our scoring language model, we use the deduplicated 1.4B parameter version of Pythia (Biderman et al., 2023), and score with it the top-20 BM25 candidates. Our model has 12 layers, hidden dimension $d = 1024$, and 8 attention heads with a head dimension of 128. We apply CCA with 2 neighbors, unless mentioned otherwise. Additional implementation details are in Appendix A and theoretical complexity of CCA layers is in Appendix B.

4 Long Range LM Datasets

We evaluate RPT on four datasets, covering domains such as books, code, and mathematical writing, which require the ability to recall information over long distances. Tab. 1 and Fig. 3 provide statistics on dataset size and the distribution over document length, showing that documents are long across all datasets and in particular PG19 and Books3, where documents typically contain 10^5 tokens or more. We briefly review the datasets.

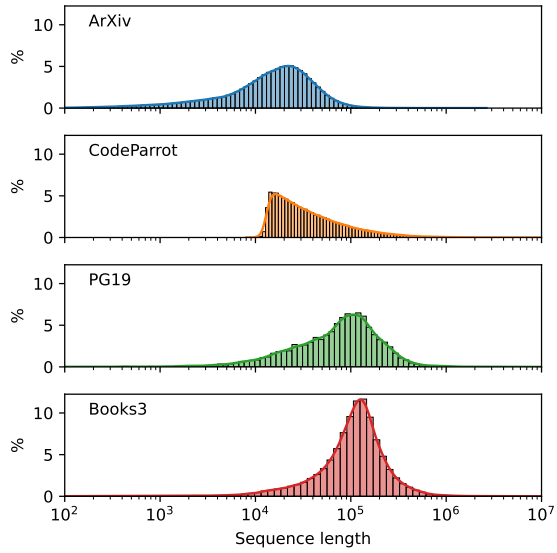

Figure 3: Histograms of the distribution over document length in tokens across all datasets. The x-axis is in log scale.

PG19 Introduced in Rae et al. (2020), PG19 is a widely-used long-range language modeling benchmark containing books from Project Gutenberg, and covering a wide range of literary genres, styles, and topics. We adopt the exact setup and data split from prior work (Wu et al., 2022; Hutchins et al., 2022; Mehta et al., 2023).

Books3 is a corpus of books released as part of the Pile (Gao et al., 2020), containing a vast collection of literary works from different domains. To our knowledge, we are the first to use this corpus as a long-range language modeling benchmark.[4]

CodeParrot (Wolf et al., 2023) is a corpus of clean, nearly-deduplicated Python code from various GitHub repositories. Modeling code requires understanding patterns and contextualizing information over long distances, making it a natural candidate for testing long-range LMs. In our experiments, we follow the approach of Wu et al. (2022), combining files from the same repository to construct a corpus with longer sequences, and create a train/test split (see Tab. 1).

ArXiv is a corpus of preprint papers extracted from ArXiv. It consists of mathematical texts that require maintaining coherence and referring to previously mentioned information over extended text. Prior work evaluated long-range LMs on this corpus (Wu et al., 2022; Hutchins et al., 2022; Mehta et al., 2023), but did not release their corpus. Thus, we use the preprocessed corpus and data splits made available by Azerbayev et al. (2023).

[4]We do not release this benchmark due to the copyright restrictions.

5 Experiments

We now turn to experiments for comparing RPT to prior work across our four datasets.

5.1 Experimental Setup

We compare to the following baselines and oracles.

Transformer-XL Our simplest baseline is a standard transformer decoder stack with sliding window attention. Put differently, we simply remove from RPT the retriever component and CCA layers in the upper decoder. Using sliding window attention (as described in §3.4) can be viewed as a variant of Transformer-XL (Dai et al., 2019). We compare RPT to Transformer-XL in multiple settings, one where we have the same number of layers and training steps for both models, and two more where we tie the number of parameters and FLOPs between the models.

RETRO We implement a modified version of Borgeaud et al. (2022), a retrieval-augmented model, where feed the top-K neighbors retrieved by BM25[5] as input to the CCA layers in the upper decoder. Concretely, Borgeaud et al. (2022) performed CCA over the representation from a separate bi-directional encoder, while our variant uses the lower-decoder representations as a replacement. This makes RPT and RETRO architectures more similar to one another and allows evaluation to center on the importance of training the retriever, which is the focus of our work. During training, we use the query $(c^{\mathbf{q}}, c^{\mathbf{t}})$, since we have access to the target chunk. During inference, we use $c^{\mathbf{q}}$.

RPT-Lex A version of RPT, where the training signal is obtained solely from lexical information, similar to TRIME (Zhong et al., 2022). Explicitly, the set of positive chunks $\mathcal{R}^{\mathbf{q}}_{\text{pos}}$ for a chunk $c^{\mathbf{q}}$ contains the top-20 chunks that have the highest BM25 score with $(c^{\mathbf{q}}, c^{\mathbf{t}})$.

RPT-Sem Our full model described in §3.

[5]Concurrent work (Doostmohammadi et al., 2023) showed that training RETRO using BM25 outperforms dense retrieval methods.

Model	ArXiv	Code	PG19	Books3	Params	Time/update
TRANSFORMER-XL (OUR IMPL.)	3.11	2.30	11.48	15.00	202M	1×
+2 LAYERS	3.07	2.26	11.2	14.52	228M	1.14×
1.5× ADDITIONAL STEPS	3.11	2.26	11.39	14.70	202M	1×
RETRO W. BM25 (OUR IMPL.)	2.94	2.17	11.44	14.60	236M	1.35×
RPT-LEX	2.92	2.23	11.59	14.32	242M	1.51×
RPT-SEM	2.77	2.17	10.96	13.91	242M	1.51×
W. 3 NEIGHBOURS	2.75	2.16	**10.92**	**13.87**		
W. 4 NEIGHBOURS	**2.74**	**2.15**	10.93	13.91		
MEMORIZING TRANSFORMER (32K)	2.92	2.18	10.97	14.40	212M	1.82×
MEMORIZING TRANSFORMER (65K)	2.93	**2.15**	10.99	14.3	212M	2.12×
BLOCK-RECURRENT TRANSFORMER	2.89	2.73	10.95	14.64	212M	1.56×
GRIFFIN	3.08	2.24	11.26	14.16	240M	1.15×
RPT-LEX W. ORACLE	2.80	2.12	10.88	13.30	242M	1.51×
RPT-SEM W. ORACLE	2.69	2.10	10.26	12.74	242M	1.51×

Table 2: Test set perplexity for all datasets along with number of parameters and the relative increase in time per update during training compared with Transformer-XL. Unless specified, models are trained for 500k steps and use 2 neighbours during inference.

Block-Recurrent Transformer We use the official training implementation[6] of Block-Recurrent Transformer (Hutchins et al., 2022) with the default configuration.

Memorizing Transformer We use the official implementation[6] of Memorizing Transformers (Wu et al., 2022), with the default configuration and a memory size of 32K and 65K tokens.

Griffin An alternative for long-range modeling is to use a hybrid of attention and linear RNNs (Orvieto et al., 2023; Gupta et al., 2023). We evaluate Griffin (De et al., 2024), a state-of-the-art model in this category. We adapt the official implementation, and supplement our Transformer-XL baseline with 5 recurrent layers in the final layers to ensure parameter parity. We use a state dimension of 2,048, and temporal dimension of 3.

Oracles For each test chunk, we can exhaustively search and use at test time the best possible neighbors for a model according to the scoring LM. This provides an upper bound for the performance of RPT-Sem, as it is trained to imitate the ranking produced by this oracle.

Metrics We use perplexity to evaluate the performance of models. In addition, we use the target score $s_{\mathbf{t}}(\cdot)$ from the scoring LM to compute for each chunk a gold ranking over all previous chunks, and to label chunks as positive/negative iff their target score is positive/negative, respectively. With this information, we can evaluate Precision@k, which is the fraction of top-k chunks according to the query-based score that are positive, and Recall@k, which is the fraction of positive chunks that are in the top-k chunks according to the query-based score. We also use the gold ranking to compute NDCG@k, which is a standard retrieval metric (Järvelin and Kekäläinen, 2002).

[6] https://github.com/google-research/meliad.

5.2 Results

Table 2 shows our main results, which show that RPT-Sem is comparable or better than all other baselines in all cases. Using a fixed retriever (RETRO) improves performance compared to Transformer-XL; RPT-Lex leads to gains in Books3 but to losses in PG19 compared to RETRO, and RPT-Sem outperforms Transformer-XL, RETRO, and RPT-Lex on ArXiv, PG19, and Books3, and has performance comparable to RETRO on CodeParrot. Even in the parameters-tied and compute-tied setting, Transformer-XL still performs substantially worse than RPT. Compared to Block-Recurrent Transformer, Memorizing Transformers and Griffin, which do not use CCA, performance is again similar or better, with significant improvements on ArXiv and Books3.

CCA enables to dynamically increase the number of neighbors at inference time. When using 3 or 4 neighbors (instead of 2), performance improves, which allows compute-performance trade-offs.

Last, oracle models consistently achieve the best perplexity across all datasets, improving from 2.74→2.69 on ArXiv, 2.15→2.10 on CodeParrot, 10.92→10.26 on PG19, and 13.87→12.74 for Books3. This shows that improving retriever train-

ing can further improve performance.

Dataset	Precision@2			Recall@10			nDCG@20		
	BM25	RPT-L	RPT-S	BM25	RPT-L	RPT-S	BM25	RPT-L	RPT-S
ArXiv	27%	26%	32%	55%	54%	58%	24%	24%	30%
Code	29%	26%	34%	53%	52%	56%	25%	23%	30%
PG19	22%	22%	28%	55%	55%	61%	18%	18%	23%
Books3	23%	19%	26%	55%	50%	58%	18%	16%	22%
Avg	25.2%	23.2%	**30.0%**	54.5%	52.7%	**58.2%**	21.2%	20.2%	**26.2%**

Table 3: Test retrieval metrics across datasets.

Retrieval metrics Table 3 presents the retrieval metrics w.r.t oracle positive chunks. Again, retrieval with RPT-Sem outperforms both RPT-Lex and BM25 in all cases. This shows the importance of training a retriever, and moreover that using semantic supervision leads to better retrieval compared to a lexical signal only.

5.3 Ablations

Tab. 4 shows the result of an ablation study over all datasets.

Only Teacher Forcing We force the model to attend to gold neighbors according to the scoring LM, without annealing p_{ss} during training. This leads to a performance drop across all datasets, and in particular for PG19 and Books3.

No Teacher Forcing Here, we do the opposite and fix $p_{\text{ss}} = 0$ throughout training, i.e., we only use the predicted neighbors and not gold ones. This can lead to undertraining of the CCA layers since they are exposed to low-quality neighbors at the beginning of training and results drop even further compared to Only Teacher Forcing.

No neighbor gating We disable neighbor gating which controls the flow of information from neighbor chunks and analyze the effect on model performance. We observe a performance reduction across all datasets, notably on Books3, where perplexity increases by 4.5 points.

Model	ArXiv	Code	PG19	Books3
RETRO W. BM25 (OUR IMPL.)	2.94	2.17	11.44	14.60
W. DPR-STYLE RETRIEVER	2.97	2.28	11.7	14.86
RPT-LEX	2.92	2.23	11.59	14.32
W. DPR-STYLE RETRIEVER	2.84	2.26	11.11	14.17
RPT-SEM	2.77	2.17	10.96	13.91
W. DPR-STYLE RETRIEVER	2.98	2.33	11.62	14.66
RPT-SEM - ONLY TEACHER FORCING	2.91	2.22	11.54	14.66
RPT-SEM - NO TEACHER FORCING	2.95	2.26	13.10	14.40
RPT-SEM - NO NEIGHBOR GATING	2.92	2.20	11.50	18.68

Table 4: Results of our ablation study.

DPR-style retriever To study the importance of joint training, we test performance when using retrievers that are trained separately from the LM, thereby inducing a train-test mismatch. We train dense retrievers using the standard DPR training procedure (Karpukhin et al., 2020) on each dataset (see Appendix C for training details), and for each of our CCA models use this retriever instead of the one it was trained with. Interestingly, we observe RPT-Lex can effectively utilize the DPR-style neighbors giving it a slight performance improvement on 3 of the 4 datasets.

As expected, the two models trained with the stronger retrievers suffer from the train-test mismatch, replacing the BM25 retriever and RPT-Sem retriever with the DPR-style retriever causes both models to suffer performance degradation on all datasets, suggesting that the non-ablated performance is the result of coordination between the retriever and the language model.

5.4 Analysis

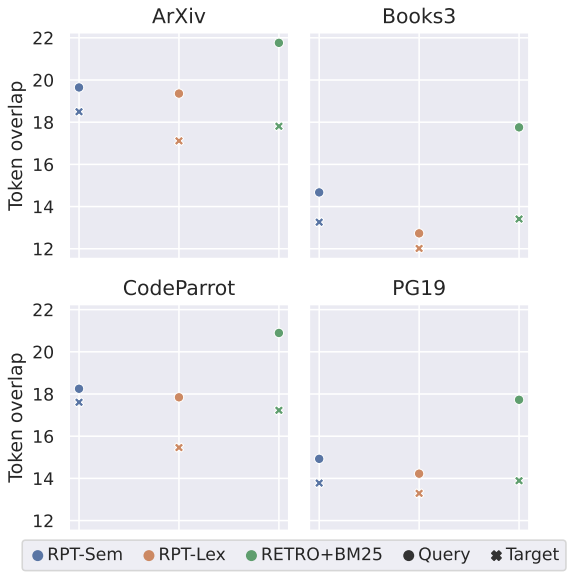

Figure 4: We measure the number of unique token overlap between query/target chunks and the best retrieved neighbor.

Token overlap Fig. 4 plots the average number of tokens that overlap between the query/target chunks in the best retrieved neighbor for RETRO, RPT-Lex, and RPT-Sem. RPT-Sem retrieves paragraphs with higher overlap with the *target* chunk compared to RPT-Lex. Naturally, BM25 retrieves chunks with the highest overlap with the *query*

chunk. However, this does not translate to higher lexical overlap for the *target* chunk.

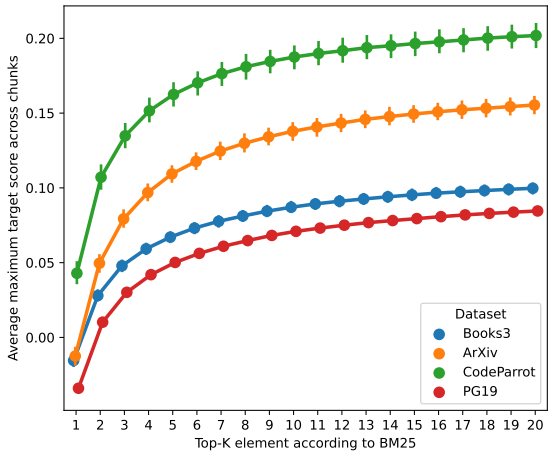

Figure 5: The maximal target score $s_{\mathbf{t}}(\cdot)$ for the top-K chunks retrieved by BM25 averaged across chunks and for all datasets. Since the maximal target score for the top-20 chunks is much higher than for the top-2, learning to rerank the top-20 BM25 candidates can lead to substantial improvements in retrieval quality.

Supervision quality We train RPT-Sem using information from the target scoring function $s_{\mathbf{t}}(\cdot)$, which we saw leads to model improvements. However, the target scoring function only provides a reranking of the top-20 candidates according to BM25. Thus, a natural question is how much does the supervision quality improve through this reranking. Fig 5 shows for every rank K the maximal target score among the top-K chunks according to BM25, averaged over chunks and across our 4 datasets. Clearly, reranking the top-20 BM25 candidates has a lot of potential, as the maximal target score is much higher for the top-20 candidates compared to the top-2. This hints that longer and better training of the retriever can further improve the performance of RPT-Sem.

Interestingly, our analysis sheds light on why RPT-Sem outperforms RETRO clearly on Books3 and PG19 but less so on CodeParrot. The maximal target score for CodeParrot when $k = 2$ is already quite high – around 0.1, which corresponds to more than 10% improvement in the probability of the target chunk compared to the local context. Conversely, for PG19 and Books3, the target score when $k = 2$ is closer to 0.

Subgroup analysis Fig 6 shows the average relative improvement (across chunks) of RETRO, RPT-Lex, and RPT-Sem compared to Transformer-XL,

Figure 6: Relative improvement with/without correct retrieval.

when distinguishing between cases where a "gold" oracle chunk was retrieved and cases where no gold chunk was retrieved.

As expected, RPT-Sem leads to improvements on all datasets, and outperforms other baselines except for RETRO on CodeParrot where performance is similar. Second, cases where a gold chunk was retrieved indeed typically lead to larger improvements, but we witness improvements even in cases where a gold chunk was not retrieved, which shows that the model can still benefit from such retrievals.

Qualitative analysis Examining retrieved chunks, we observe that the RPT retriever is highly contextual. When applied on code, it retrieves function definitions, variable assignments, etc., on ArXiv it retrieves definitions of lemmas, theorems, etc. Fig. 7 shows an example, where we give the codebase used for this paper as input to our model and present an example query chunk where RPT produces better retrieval than BM25. We observe that the preceding context allows RPT to effectively retrieve a relevant object definition, leading to lower loss.

6 Discussion and Related Work

Relation to Fusion-in-Decoder RPT shares similarities with Fusion-in-Decoder (FiD) (Izacard and Grave, 2021b; Ivgi et al., 2023). While both RPT and FiD employ cross-attention mechanisms to integrate the retrieved context within their models, they differ in two ways. (a) In FiD, retrieval is per-

```
@flax.struct.dataclass
class FlaxRPTRetrieverEncodedOutput(ModelOutput):
    original_hidden_states: jnp.ndarray = None
    encoded_hidden_states: jnp.ndarray = None
    attention_mask: jnp.ndarray = None
    key_chunks: jnp.ndarray = None
    query_chunks: jnp.ndarray = None
    chunk_mask: jnp.ndarray = None
    ...
class FlaxRPTModule(nn.Module):
  ...
 def __call__(...
        ...
        hidden_states = self.ln_f(hidden_states)
        if not return_dict:
            return (hidden_states,) + upcoder_outputs + lowcoder_outputs
        return FlaxRPTModelOutput(
            last_hidden_state=upcoder_outputs.last_hidden_state,
            upcoder_hidden_states=upcoder_outputs.hidden_states,
            upcoder_attentions=upcoder_outputs.attentions,
            lowcoder_last_hidden_state=lowcoder_outputs.last_hidden_state,
            ...)
 ...
def forward_loglikelihood(params, rng, batch, memory):
    ...
    outputs, lowcoder_state =  _forward_loglikelihood_lowcoder(params, rng, batch)
    if 'cache' in lowcoder_state:
        params['cache'] = lowcoder_state['cache']
    outputs = jax.tree_map(lambda x: jax.device_get(x).astype(np.float32), outputs)
    neighbor_hidden_states, neighbor_mask, *_ = memory.add(
                        input_tokens=batch["input_tokens"],
                        encoded_hidden_states=outputs.encoded_hidden_states,
                        key_chunks=outputs.key_chunks,
                        query_chunks=outputs.query_chunks,
                             )
         ...
```

Figure 7: An illustrative example showcasing the top-1 retrieved neighbors for both **RPT-Sem** and **BM25** models applied to RPT's code. The variable `outputs` in the **query chunk** is a member of the class `FlaxRPTRetrieverEncodedOutput`. **RPT-Sem** successfully retrieves the object's definition leading to a reduced loss on the **target chunk**, in comparison to **BM25**.

formed only once based on the initial prompt/query, while RPT continuously performs retrieval at the chunk level throughout generation. (b) FiD encodes retrieved neighbors separately using a bidirectional encoder and only then applies cross-attention in the decoder. In RPT, the decoder computes chunk embeddings and performs native retrieval, and then chunked cross-attention is applied to fuse the retrieved context with the model's predictions. We view RPT, which uses lower-decoder encodings, as more natural in the context of continuous generation (e.g., chatbots or agents), since the model generates representations and uses them later as keys, and thus generating retrieval representations bears zero cost.

Long-range language modeling A primary focus in long-range language modeling has been addressing the quadratic complexity of attention in order to develop more efficient mechanisms for handling long texts. For instance, Transformer-XL (Dai et al., 2019) processes the input using a segment-level mechanism while retaining a cache from previous segments. Longformer (Beltagy et al., 2020) extends this idea to accommodate even longer contexts. Several works previously viewed retrieval as a long-range problem. Memorizing Transformers (Wu et al., 2022) employed a single k-NN layer and retrieve cached keys and values, but they do not back-propagate gradients through the sparse retrieval operation. Similarly, Bertsch et al. (2023) demonstrated that this approach can

be used with any existing pre-trained model and applied it at every attention layer for long summarization tasks. From an analysis perspective, past work (Press et al., 2021) demonstrated that standard LM benchmarks are not ideal for measuring the long-range capabilities of models. Sun et al. (2021) discuss various types of sequences that benefit from having a long context, and Rae and Razavi (2020) investigate long-range architectural choices and recommend increasing long-range capabilities in the upper layers.

Efficient language modeling Sparse strategies, such as those proposed in Zaheer et al. (2020); Roy et al. (2021); Kitaev et al. (2020), similarly to RPT, attend to only a subset of tokens through clustering or hashing methods, which are trained by propagating gradients through the sparse operation. In RPT, sparsity is due to the retriever top-K operation, which is trained using high-quality supervision from a reference language model. Another approach for efficiently modeling long text involves compressing the input and attending over the compressed sequence (Martins et al., 2022; Rae et al., 2020), or learning to ignore irrelevant tokens (Sukhbaatar et al., 2021). However, empirically most efficient transformer architectures trade off efficiency for quality. Recently, state-space models (Mehta et al., 2023; Gu and Dao, 2023; Fu et al., 2023) models emerged as an efficient alternative, which approaches Transformer quality. In this paper, we explore models that are based on classic quadratic Transformer. We argue that the underlying model is orthogonal to our contribution and can be replaced by other efficient alternatives and combined with retrieval. We leave this exploration for future work.

Retrieval augmented LMs Retrieval-augmented LMs have emerged as a prominent approach for efficiently leveraging external knowledge while generating text. These models can be broadly divided into those operating at token-level granularity and those operating at sequence-level granularity. Token-level methods, such as kNN-LM (Khandelwal et al., 2020), TRIME (Zhong et al., 2022), and SPALM (Yogatama et al., 2021), retrieve information for individual tokens. Sequence-level approaches like RAG (Lewis et al., 2020) utilize pre-trained encoder-decoder models with pre-trained retrievers for tasks like open-domain question answering. Similarly, FiD (Izacard and Grave, 2021b) employs generative encoder-decoder models that fuse evidence from multiple passages during the decoding process, closely related to the CCA mechanism. Recently, Wang et al. (2023) demonstrated the potential benefits of conducting retrieval and chunked cross-attention at each time step, compared with the original RETRO (Borgeaud et al., 2022) paper, which retrieves every $m = 64$ steps.

Joint retriever-reader training Joint training approaches typically concentrate on transferring information between a pre-trained reader into a pre-trained retriever. These methods commonly involve updating the retriever index during the training process in the context of knowledge-intensive tasks, such as open-domain question answering. For instance, REALM (Guu et al., 2020) utilizes masked language modeling as a learning signal to update the retriever. EMDR2 (Sachan et al., 2021) extends FiD by using encoder-decoder models to back-propagate errors from the predicted answer to the retriever. Similarly, Izacard and Grave (2021a); Jiang et al. (2022) uses attention scores from the reader to supervise the retriever directly using the attention matrix as a training signal to enable joint end-to-end training with the supervision of the downstream task. Notably, Izacard et al. (2022b) further scale up these approaches and jointly train a retriever with an encoder-decoder model, demonstrating strong few-shot learning capabilities. They also investigate various retriever updating techniques to address train-test mismatches in the retrieval process. We do not encounter the issue of index update since we compute the entire index through a forward pass.

Retriever Pre-training Early work on retriever pre-training relied on the unsupervised Inverse Cloze Task to pre-train the retriever (Lee et al., 2019; Guu et al., 2020). It was later shown that directly using BERT (Devlin et al., 2019) with a supervised objective is sufficient to get good performance on standard benchmarks (Karpukhin et al., 2020). However, this paradigm showed lackluster performance on long-tail entities compared to BM25 (Amouyal et al., 2023; Sciavolino et al., 2021). Recently, unsupervised pre-training methods (Gao and Callan, 2022; Ram et al., 2022; Izacard et al., 2022a) enabled improved performance. However, these methods are initialized from a pre-trained BERT (Devlin et al., 2019) encoder model, while RPT is a retriever-reader architecture trained

from scratch that outperforms BM25 without any additional pre-training.

Supervising retrievers with LLMs EPR (Rubin et al., 2022) demonstrated that LLMs could be employed to train a retriever for prompt retrieval by estimating the probability of an output given the input and a candidate training example as the prompt. Similar techniques were applied to open-domain question answering via re-ranking retrieval results (Sachan et al., 2022; Ram et al., 2023) and to supervise retrievers through perplexity distillation (Izacard et al., 2022b). Recently, Shi et al. (2024) utilized this supervision method to improve the performance of various LLMs in a black-box fashion.

7 Conclusion

In this work, we present the Retrieval-Pretrained Transformer (RPT), a retrieval-augmented LM where the retriever is trained as a native component of the LM to retrieve semantically relevant chunks for future text prediction. We evaluate RPT on four long-range language modeling tasks, including books, code, and mathematical writing. We demonstrate that by seamlessly integrating the retriever into the architecture and training process, RPT benefits from the fusion of retrieved context, improving over strong retrieval-augmented baselines. While this work focuses on retrieval from long texts, we argue our empirical findings show that adapting our procedure for general web-based corpora retrieval is an exciting future direction. This will require overcoming technical difficulties related to scaling and pretraining corpus construction. We envision RPT will pave the way for a new generation of pretrained language models with retrieval deeply integrated throughout their architecture and training process.

Acknowledgments

This research was supported with Cloud TPUs from Google's TPU Research Cloud (TRC) and The European Research Council (ERC) under the European Union Horizons 2020 research and innovation programme (grant ERC DELPHI 802800). Ohad would like to thank Iz Beltagy for suggesting the TRC program, and the entire TAU NLP lab and especially Guy Dar and Itay Itzhak. This work was completed in partial fulfillment of the Ph.D. degree of Ohad Rubin.

References

Samuel Amouyal, Tomer Wolfson, Ohad Rubin, Ori Yoran, Jonathan Herzig, and Jonathan Berant. 2023. QAMPARI: A benchmark for open-domain questions with many answers. In *Proc. of the Third Workshop on GEM. ACL*.

Zhangir Azerbayev, Edward Ayers, and Bartosz Piotrowski. 2023. Proof-Pile: A Pre-training Dataset of Mathematical Text.

Iz Beltagy, Matthew E. Peters, and Arman Cohan. 2020. Longformer: The long-document transformer.

Samy Bengio, Oriol Vinyals, Navdeep Jaitly, and Noam Shazeer. 2015. Scheduled sampling for sequence prediction with recurrent neural networks. In *Proc. of NeurIPS*.

Amanda Bertsch, Uri Alon, Graham Neubig, and Matthew R. Gormley. 2023. Unlimiformer: Long-range transformers with unlimited length input. In *Proc. of NeurIPS*.

Stella Biderman, Hailey Schoelkopf, Quentin Anthony, Herbie Bradley, Kyle O'Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, and Oskar van der Wal. 2023. Pythia: A suite for analyzing large language models across training and scaling.

Sidney Black, Stella Biderman, Eric Hallahan, Quentin Anthony, Leo Gao, Laurence Golding, Horace He, Connor Leahy, Kyle McDonell, Jason Phang, Michael Pieler, Usvsn Sai Prashanth, Shivanshu Purohit, Laria Reynolds, Jonathan Tow, Ben Wang, and Samuel Weinbach. 2022. GPT-NeoX-20B: An open-source autoregressive language model. In *Proc. of the BigScience Workshop*.

Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George van den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, Diego de Las Casas, Aurelia Guy, Jacob Menick, Roman Ring, Tom Hennigan, Saffron Huang, Loren Maggiore, Chris Jones, Albin Cassirer, Andy Brock, Michela Paganini, Geoffrey Irving, Oriol Vinyals, Simon Osindero, Karen Simonyan, Jack W. Rae, Erich Elsen, and Laurent Sifre.

2022. Improving language models by retrieving from trillions of tokens. In *Proc. of ICML*.

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language models are few-shot learners. In *Proc. of NeurIPS*.

Christopher J. C. Burges, Robert Ragno, and Quoc Viet Le. 2006. Learning to rank with non-smooth cost functions. In *Proc. of NeurIPS*.

Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayana Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. 2022. Palm: Scaling language modeling with pathways.

Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc Le, and Ruslan Salakhutdinov. 2019. Transformer-XL: Attentive language models beyond a fixed-length context. In *Proc. of ACL*.

Soham De, Samuel L. Smith, Anushan Fernando, Aleksandar Botev, George Cristian-Muraru, Albert Gu, Ruba Haroun, Leonard Berrada, Yutian Chen, Srivatsan Srinivasan, Guillaume Desjardins, Arnaud Doucet, David Budden, Yee Whye Teh, Razvan Pascanu, Nando De Freitas, and Caglar Gulcehre. 2024. Griffin: Mixing gated linear recurrences with local attention for efficient language models.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proc. of NAACL-HLT*.

Ehsan Doostmohammadi, Tobias Norlund, Marco Kuhlmann, and Richard Johansson. 2023. Surface-based retrieval reduces perplexity of retrieval-augmented language models. In *Proc. of ACL*.

Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2024. The faiss library.

William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *J. Mach. Learn. Res.*, 23:1–39.

Daniel Y Fu, Tri Dao, Khaled Kamal Saab, Armin W Thomas, Atri Rudra, and Christopher Re. 2023. Hungry hungry hippos: Towards language modeling with state space models. In *Proc. of ICLR*.

Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. 2020. The pile: An 800gb dataset of diverse text for language modeling.

Luyu Gao and Jamie Callan. 2022. Unsupervised corpus aware language model pre-training for dense passage retrieval. In *Proc. of ACL*.

Albert Gu and Tri Dao. 2023. Mamba: Linear-time sequence modeling with selective state spaces.

Ankit Gupta, Harsh Mehta, and Jonathan Berant. 2023. Simplifying and understanding state space models with diagonal linear rnns.

Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. 2020. Realm: Retrieval-augmented language model pre-training. In *Proc. of ICML*.

Yangsibo Huang, Daogao Liu, Zexuan Zhong, Weijia Shi, and Yin Tat Lee. 2023. $k$nn-adapter: Efficient domain adaptation for black-box language models.

DeLesley Hutchins, Imanol Schlag, Yuhuai Wu, Ethan Dyer, and Behnam Neyshabur. 2022. Block-recurrent transformers. In *Proc. of NeurIPS*.

Maor Ivgi, Uri Shaham, and Jonathan Berant. 2023. Efficient Long-Text Understanding with Short-Text Models. In *Transactions of the Association for Computational Linguistics*, volume 11, pages 284–299.

Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave. 2022a. Unsupervised dense information retrieval with contrastive learning. *Transactions on Machine Learning Research*.

Gautier Izacard and Edouard Grave. 2021a. Distilling knowledge from reader to retriever for question answering. In *Proc. of ICLR*.

Gautier Izacard and Edouard Grave. 2021b. Leveraging passage retrieval with generative models for open domain question answering. In *Proc. of EACL*.

Gautier Izacard, Patrick Lewis, Maria Lomeli, Lucas Hosseini, Fabio Petroni, Timo Schick, Jane Dwivedi-Yu, Armand Joulin, Sebastian Riedel, and Edouard Grave. 2022b. Atlas: Few-shot learning with retrieval augmented language models. *J. Mach. Learn. Res.*, 24:1–43.

Kalervo Järvelin and Jaana Kekäläinen. 2002. Cumulated gain-based evaluation of ir techniques. *ACM Transactions on Information Systems*, 20:422–446.

Zhengbao Jiang, Luyu Gao, Zhiruo Wang, Jun Araki, Haibo Ding, Jamie Callan, and Graham Neubig. 2022. Retrieval as attention: End-to-end learning of retrieval and reading within a single transformer. In *Proc. of EMNLP*.

Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense passage retrieval for open-domain question answering. In *Proc. of EMNLP*.

Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis. 2020. Generalization through memorization: Nearest neighbor language models. In *Proc. of ICLR*.

Nikita Kitaev, Lukasz Kaiser, and Anselm Levskaya. 2020. Reformer: The efficient transformer. In *Proc. of ICLR*.

Kenton Lee, Ming-Wei Chang, and Kristina Toutanova. 2019. Latent retrieval for weakly supervised open domain question answering. In *Proc. of ACL*.

Patrick S. H. Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Proc. of NeurIPS*.

Pedro Henrique Martins, Zita Marinho, and Andre Martins. 2022. ∞-former: Infinite memory transformer. In *Proc. of ACL*.

Harsh Mehta, Ankit Gupta, Ashok Cutkosky, and Behnam Neyshabur. 2023. Long range language modeling via gated state spaces. In *Proc. of ICLR*.

Antonio Orvieto, Samuel L Smith, Albert Gu, Anushan Fernando, Caglar Gulcehre, Razvan Pascanu, and Soham De. 2023. Resurrecting recurrent neural networks for long sequences. In *Proc. of ICML*.

Ofir Press, Noah A. Smith, and Mike Lewis. 2021. Shortformer: Better language modeling using shorter inputs. In *Proc. of ACL*.

Ofir Press and Lior Wolf. 2017. Using the output embedding to improve language models. In *Proc. of EACL*.

Jack Rae and Ali Razavi. 2020. Do transformers need deep long-range memory? In *Proc. of ACL*.

Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, Chloe Hillier, and Timothy P. Lillicrap. 2020. Compressive transformers for long-range sequence modelling. In *Proc. of ICLR*.

Ori Ram, Yoav Levine, Itay Dalmedigos, Dor Muhlgay, Amnon Shashua, Kevin Leyton-Brown, and Yoav Shoham. 2023. In-context retrieval-augmented language models. *Transactions of the Association for Computational Linguistics*, 11:1316–1331.

Ori Ram, Gal Shachaf, Omer Levy, Jonathan Berant, and Amir Globerson. 2022. Learning to retrieve passages without supervision. In *Proc. of NAACL-HLT*.

Stephen Robertson and Hugo Zaragoza. 2009. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends in Information Retrieval*, 3:333–389.

Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. 2021. Efficient content-based sparse attention with routing transformers. *Transactions of the Association for Computational Linguistics*, 9:53–68.

Ohad Rubin, Jonathan Herzig, and Jonathan Berant. 2022. Learning to retrieve prompts for in-context learning. In *Proc. of NAACL-HLT*.

Devendra Sachan, Mike Lewis, Mandar Joshi, Armen Aghajanyan, Wen-tau Yih, Joelle Pineau, and Luke Zettlemoyer. 2022. Improving passage retrieval with zero-shot question generation. In *Proc. of EMNLP*.

Devendra Singh Sachan, Siva Reddy, William L. Hamilton, Chris Dyer, and Dani Yogatama. 2021. End-to-end training of multi-document reader and retriever for open-domain question answering. In *Proc. of NeurIPS*.

Christopher Sciavolino, Zexuan Zhong, Jinhyuk Lee, and Danqi Chen. 2021. Simple entity-centric questions challenge dense retrievers. In *Proc. of EMNLP*.

Weijia Shi, Sewon Min, Michihiro Yasunaga, Minjoon Seo, Rich James, Mike Lewis, Luke Zettlemoyer, and Wen tau Yih. 2024. Replug: Retrieval-augmented black-box language models. In *Proc. of NAACL-HLT*.

Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. 2024. Roformer: Enhanced transformer with rotary position embedding. *Neurocomput.*, 568.

Sainbayar Sukhbaatar, Da Ju, Spencer Poff, Stephen Roller, Arthur Szlam, Jason Weston, and Angela Fan. 2021. Not all memories are created equal: Learning to forget by expiring. In *Proc. of ICML*.

Simeng Sun, Kalpesh Krishna, Andrew Mattarella-Micke, and Mohit Iyyer. 2021. Do long-range language models actually use long-range context? In *Proc. of EMNLP*.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. Llama: Open and efficient foundation language models.

Boxin Wang, Wei Ping, Peng Xu, Lawrence McAfee, Zihan Liu, Mohammad Shoeybi, Yi Dong, Oleksii Kuchaiev, Bo Li, Chaowei Xiao, Anima Anandkumar, and Bryan Catanzaro. 2023. Shall we pretrain autoregressive language models with retrieval? a comprehensive study. In *Proc. of EMNLP*.

Thomas Wolf, Loubna Ben Allal, Leandro von Werra, Li Jia, and Armel Zebaze. 2023. A dataset of python files from github.

Yuhuai Wu, Markus Norman Rabe, DeLesley Hutchins, and Christian Szegedy. 2022. Memorizing transformers. In *Proc. of ICLR*.

Dani Yogatama, Cyprien de Masson d'Autume, and Lingpeng Kong. 2021. Adaptive semiparametric language models. *Transactions of the Association for Computational Linguistics*, 9:362–373.

Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontañón, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. 2020. Big bird: Transformers for longer sequences. In *Proc. of NeurIPS*.

Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin,

Todor Mihaylov, Myle Ott, Sam Shleifer, Kurt Shuster, Daniel Simig, Punit Singh Koura, Anjali Sridhar, Tianlu Wang, and Luke Zettlemoyer. 2022. Opt: Open pre-trained transformer language models.

Zexuan Zhong, Tao Lei, and Danqi Chen. 2022. Training language models with memory augmentation. In *Proc. of EMNLP*.

Juntang Zhuang, Tommy Tang, Yifan Ding, Sekhar C. Tatikonda, Nicha C. Dvornek, Xenophon Papademetris, and James S. Duncan. 2020. Adabelief optimizer: Adapting stepsizes by the belief in observed gradients. In *Proc. of NeurIPS*.

A Additional Implementation Details

Models are implemented in JAX with a dropout rate of 0.05, and the AdaBelief (Zhuang et al., 2020) optimizer with a weight decay of 1e-8, cosine decay to 0.1 of max learning rate, global gradient norm clipping of 1, and tied input embedding (Press and Wolf, 2017). Grid search determined τ values: 128 for Books3, 4 for PG19, 2 for CodeParrot, and 8 for ArXiv. We set $\alpha_{\text{ret}} = 1e-9$ for all datasets and a base learning rate of $5e-3$, using the validation set for hyperparameter selection.

B Computational Complexity

The per token computational complexity of an attention layer in a transformer model with dimension d, $|Q|$ queries and $|K|$ keys is $2 \cdot d \cdot (|K| \cdot |Q| + |K| \cdot d + |Q| \cdot d)$ flops.[7] By setting $N = |Q| = |K|$ and adding the cost the feed-forward layer, we get that the per token cost for a transformer block when $d \gg N$ is $2d(N + 2d) + 8d^2 \approx 12d^2$ flops. For CCA, the cost is dependent on the chunk size C, and number of neighbors k. Setting $|K| = 2Ck$ and $|Q| = C$, and assuming $d \gg Ck$, the cost per token for a CCA layer is $2d(2Ck + 2dk + d) \approx (4k + 2) \cdot d^2$ flops. Our per token overhead for $\alpha \in [0, 1]$ of the blocks including CCA is $\approx \alpha(\frac{k}{3} + \frac{1}{6})$. In our experiments, we use CCA in 5 of the 12 layers so $\alpha = \frac{5}{12}$ and $k = 2$, and get that CCA contributes an overhead of approximately 1.29x. Using similar logic, the constant cost for the retriever component is the two linear projections, the two additional bidirectional attention layers, and the query augmentation layer resulting in $\frac{1}{n_{\text{layers}}} \cdot (\frac{7k}{6} + \frac{1}{2})$, or a final overhead of 1.49x which is in line with our effective measured runtime overhead of 1.51x (see Table 2).

[7] For a query matrix $Q \in \mathbb{R}^{|Q| \times d}$ and a key/value matrix $K \in \mathbb{R}^{|K| \times d}$, it consists of the following operations: multiplication with W_Q, W_K, and W_V for the queries, keys, and values, each costing $|Q| \cdot d^2$, $|K| \cdot d^2$, and $|K| \cdot d^2$ flops respectively. Computing the attention matrix and multiplying it by the values each requires $|Q| \cdot |K| \cdot d$ flops. Finally, multiplying by the output matrix is an additional $|Q| \cdot d^2$ flops.

C DPR-style retriever training details

We followed the training recipe of DPR (Karpukhin et al., 2020) in training a BERT-base retriever with contrastive loss. The DPR objective requires positive and hard negatives to converge successfully, and here we use the top-1 scoring BM25 chunk as the positive example and the chunk ranked 5th by BM25 as the hard negative example. To ensure a fair comparison, we train our contrastive retriever on 16x more examples than the original DPR recipe describes.